An Introduction to Running JavaScript in Java with Nashorn

This 'teaser' article is intended to whet the appetite of the reader to learn more. Nashorn is a JavaScript engine developed by Oracle. It aims at allowing JavaScript to be embedded in Java, and to develop standalone JavaScript applications.

JavaScript is one of the fastest growing and most widely adopted languages today. Having started out as a language to provide interactivity in a browser, it has grown multi-fold. In 2009, Node.js brought it to server side programming. And in 2013, it came to Windows desktops when Microsoft released the Metro (or Modern) apps, which allowed developers to write desktop apps using JavaScript.

Fast forward to 2016 and JavaScript is everywhere! With new language features, a huge list of libraries and an ever growing community, it's getting bigger by the day.

Introducing Nashorn

The JavaScript language needs to be executed on a JavaScript engine. Some very popular engines out there are V8 (Chrome and Node), Chakra (Microsoft Edge) and Spider Monkey (Mozilla Firefox). Similarly, Nashorn is the JavaScript engine in Java, which enables embedding JavaScript in Java applications.

Up and running with Nashorn

Cutting right to the chase, there are two ways in which you can execute JavaScript in Java:

1. Using the JIS command line tool
2. Directly inside Java code, using the native JavaScript engine

Using the JJS tool

JJS is a REPL (Read Evaluate Print Loop) tool inbuilt into JDK. Running JavaScript code with it is as simple as navigating to the *JAVA_HOME\bin* directory and executing the *jjs.exe* or *jjs.sh* file.

Here is an example.

```
jjs> var x = 'Hello OSFY';
jjs> print(x);
Hello OSFY
```

Using the Java API

The general purpose of executing JavaScript in a Java environment is to create a *ScriptEngine* object, and pass your JavaScript to the script engine to evaluate. So the *Hello World* code will be:

```
package com.metalop.nashorn.tutorials;
import javax.script.ScriptEngine;
import javax.script.ScriptEngineManager;
import javax.script.ScriptException;

public class HelloWorld {
            public static void main(String[] args) throws
ScriptException {
                        ScriptEngine engine = new
ScriptEngineManager().getEngineByName("nashorn");
                        engine.eval("print('Hello
World!');");
            }
}
```

Some of the shortcomings of Nashorn

- Nashorn is just a JavaScript engine, and doesn't support any of the DOM (Document Object Model) level APIs supported by browsers. So you can't use things like *console.log* statements or AJAX requests and any DOM dependent libraries like jQuery inside Nashorn.
- Nashorn currently supports only ECMA Script 5.1, and therefore you can't write JS with the ES6 and ES2017 syntax. (Later in the article, we will look at how to fix this using *Babel.js*.)

Continued on Page 75...